

Multitenant Data Platforms

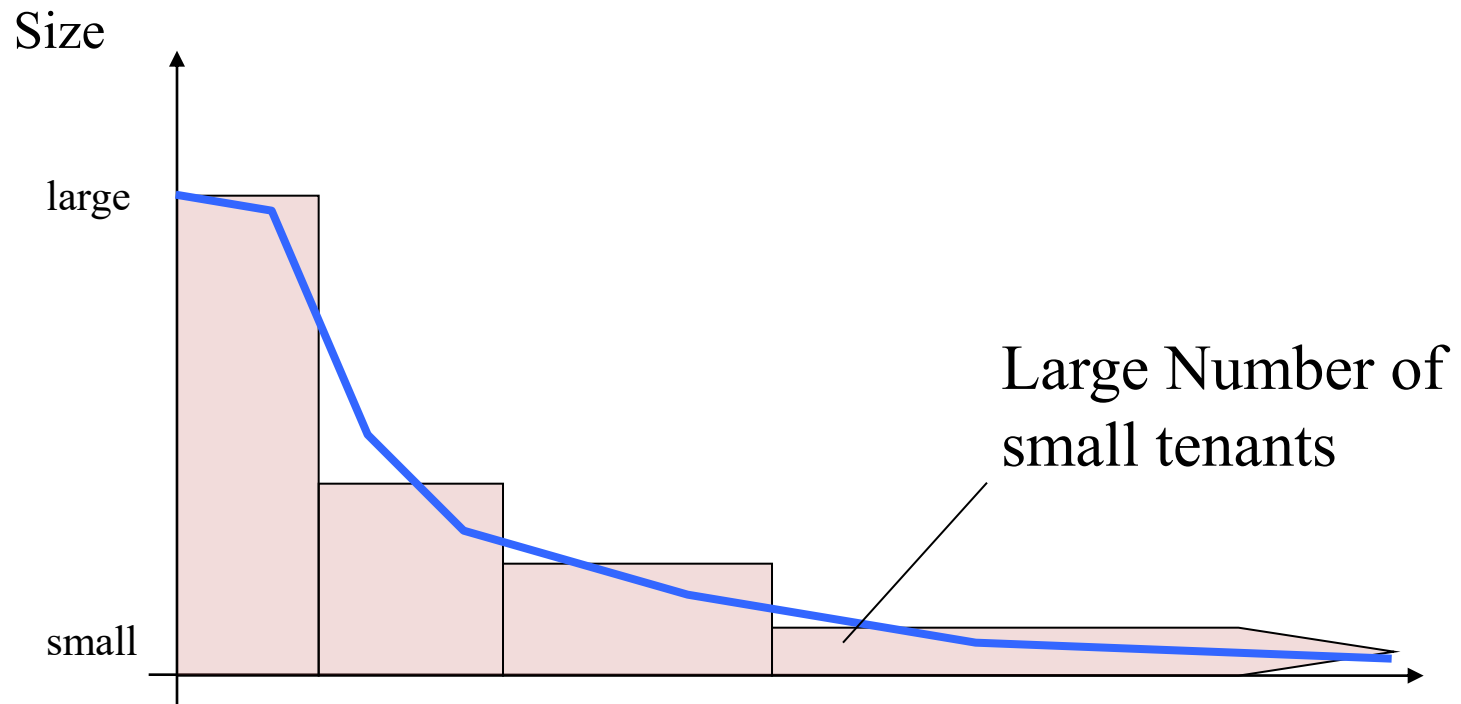
Multitenancy

- Multi-tenancy is a paradigm in which a service provider hosts **multiple clients** (tenants) on a **single shared** stack of **software** and **hardware**
- Virtualization – Multitenancy in the hardware layer
 - Major enabling technology for cloud infrastructure
- **Virtualization in the database tier**

Multitenancy

- Traditionally, the application is developed for use by a single enterprise, where products are independently deployed in the data center of the customer.
- The data created and accessed by such applications usually belong to one organization.
- Hosted SaaS platforms require a single application code to run on data of multiple customers (tenants). Such behaviour is called multi-tenancy.

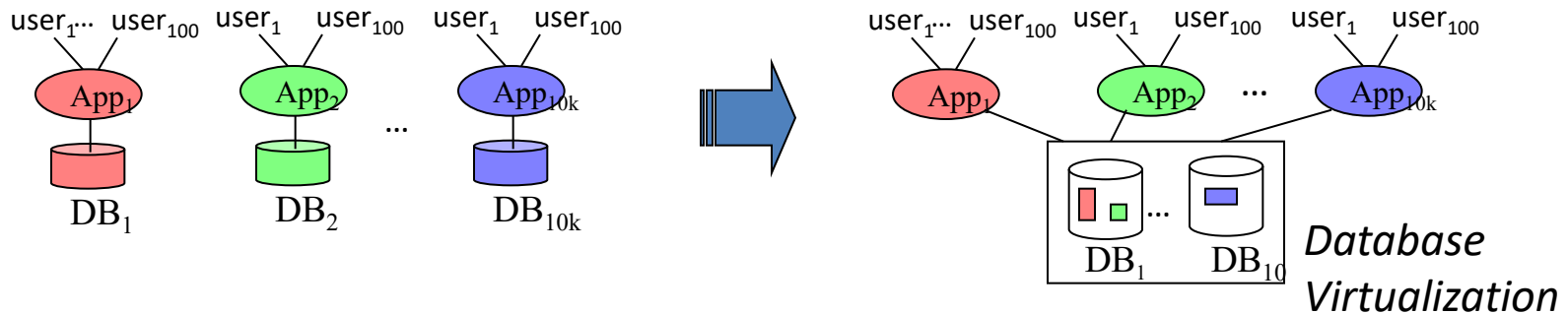
Capturing the “Long Tail” in Multi-tenant Applications



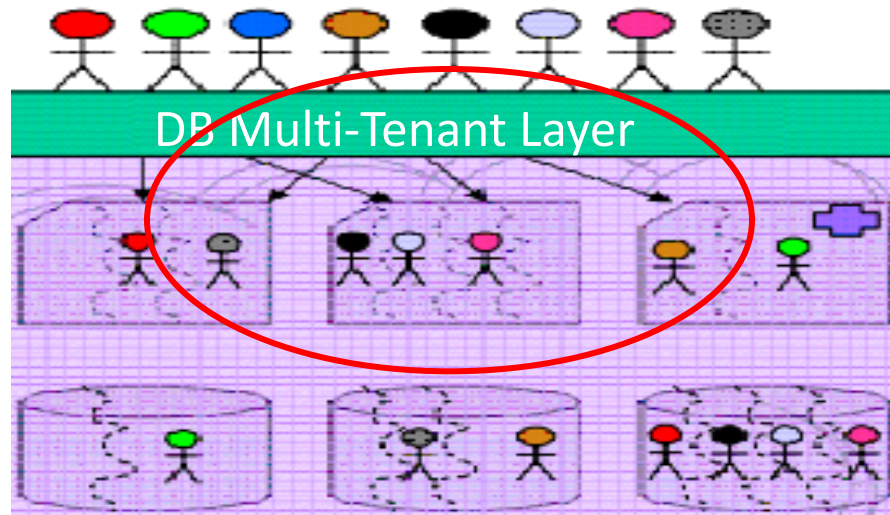
Multi Application vs. Multi-tenant Application Scenario

- **Multi Application Scenario**

—Support a very large number of database applications (with different schemas)

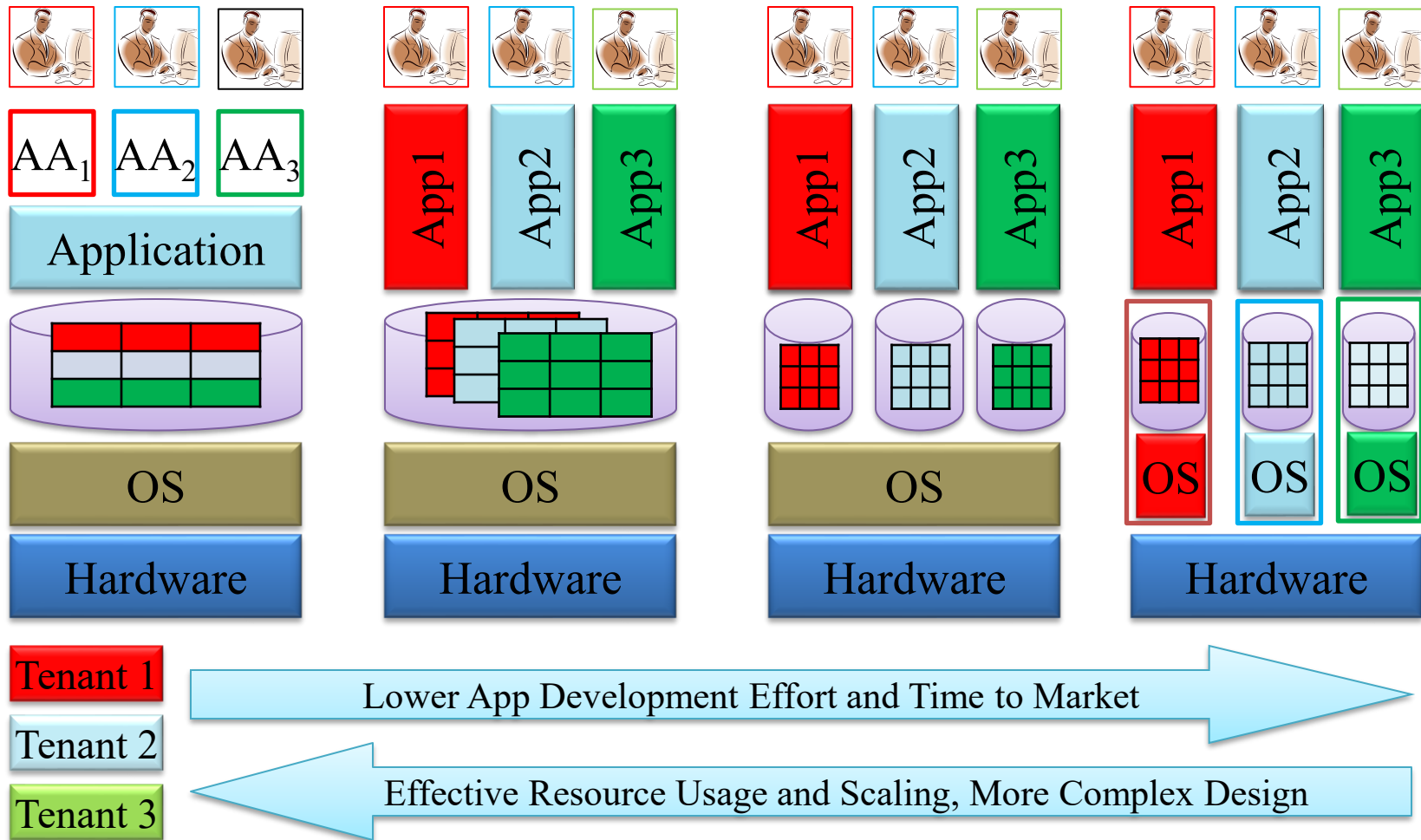


Multi-tenancy Challenges



Isolation, Scalability, Performance,
Customization, Resource Utilization,
Metering ...

Multitenancy Trade-offs



Multi-tenancy

Resource Sharing and Isolation

MT Sharing Model	Isolation	Description
None	none	Tenants are on different machines. No Sharing
<i>Shared Hardware</i>	VM	Tenants are on the same hardware but isolated in different virtual machines
Shared VM	OS User	Tenants are on the same virtual machine but isolated by OS user authentication (OS level protection)
Shared OS level	DB instance	Tenants share the OS but have different DB instances
<i>Shared DB Instance</i>	DB	Tenants are in the same DB instance but isolated using different databases
<i>Shared Table</i>	Row	Tenants are in the same tables but isolated by row level security

Slides adapted from a presentation by B. Reinwald

Multitenancy Trade-offs

	Isolated Databases	Separate Schemas	Shared Tables
Simplicity	simple	simple (but need naming and mapping schemes)	hard
Customizability (schema)	high	high	low
Rigorous Isolation (regulatory law)	best	moderate	lowest
Resource Cost/tenant	high	low	lowest
#Tenants	Low	large	Largest

Multitenancy Trade-offs

	Isolated Databases	Separate Schemas	Shared Tables
Tools	tools to deal w/ large number of DBs	tools to deal w/ large number of tables	n/a
DB implementation cost	Lowest (query routing and simple mapping layer)	Low (query routing, simple mapping layer and query mapping)	High (query routing, simple mapping layer, query mapping, row-level isolation)
Scalability	Per tenant	Need some data/load balancing w/ dynamic migration	Need some data/load balancing w/ dynamic migration
Query Optimization	Less critical	Less critical	Critical (wrong plan over very large tables is disastrous)
Per Tenant Query Performance	As usual	need query governance	Need query governance and tenant-specific statistics

Force.com architecture

Shared table approach [Weissman et al., SIGMOD 2009]

- **Metadata driven architecture**
 - use a runtime engine that generates application components from metadata
 - clear separation of the compiled runtime engine (kernel), application data, the metadata that describes the base functionality of an application, and the metadata that corresponds to each tenant's data and customizations.
- Tenant specific customizations information stored as **metadata**
- Engine uses metadata to generate **virtual application components** at runtime
 - Metadata is key – **cache metadata**
- Application data stored in a large shared table – referred to as the **heap**
 - **Materialize** some virtual tables
- **Pivot tables** used for **indexing**, maintaining relationships, **uniqueness constraints**
 - A collection of pivot tables used

Shared table design

- The heap stores all application data
 - Generic schema – **flex columns**
 - Native database index and query processing cannot be applied directly
- Metadata used to interpret data from the heap
- Application server logic for data re-mapping
- Strongly typed pivot tables act as index
- Advanced optimization techniques such as **chunk folding** proposed [Aulbach et al, SIGMOD 2008]

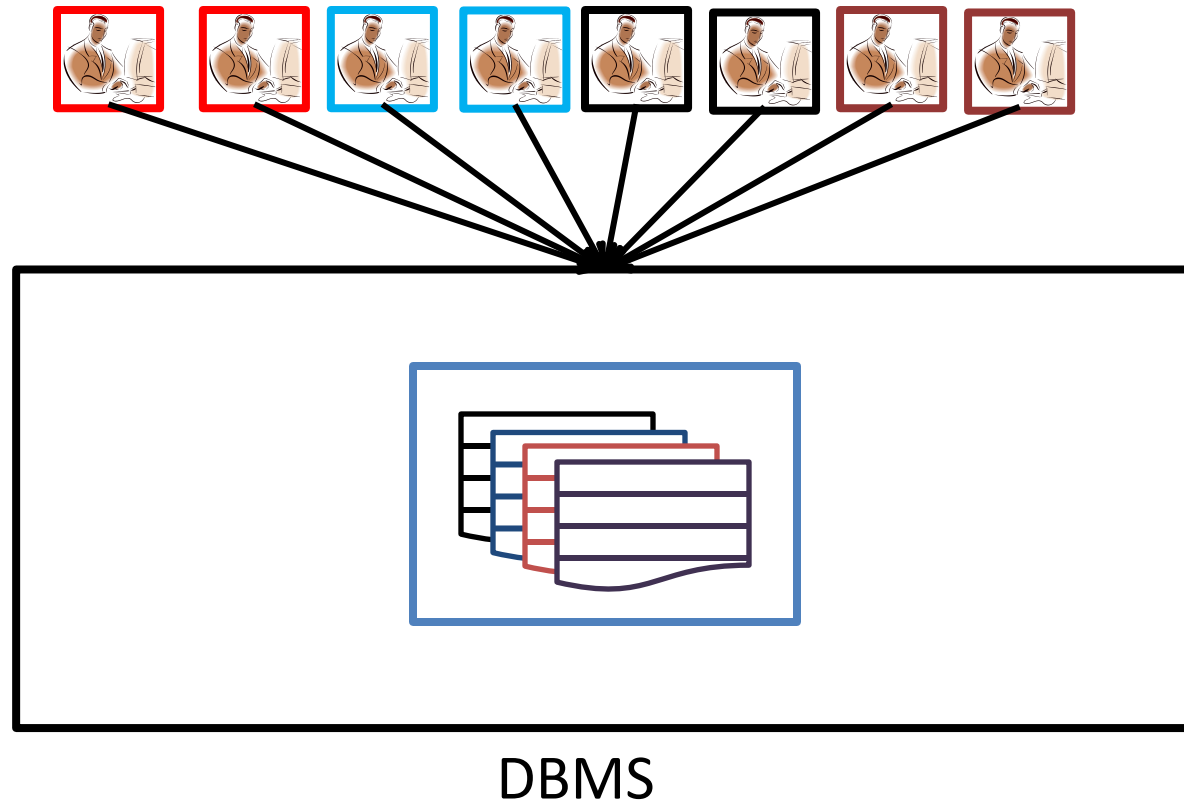
Supporting Large Number of Small Applications [Yang et al., CIDR 2009]

- “**Small**” applications $\Rightarrow$ data fits into a single machine
- Each tenant stored in a single MySQL instance
- Use **shared-nothing** MySQL installation
- Build the **distributed control fabric**
 - Query routing
 - Failure detection and Load balancing
 - Guaranteeing SLAs
- Similar to the shared process abstraction

Elasticity

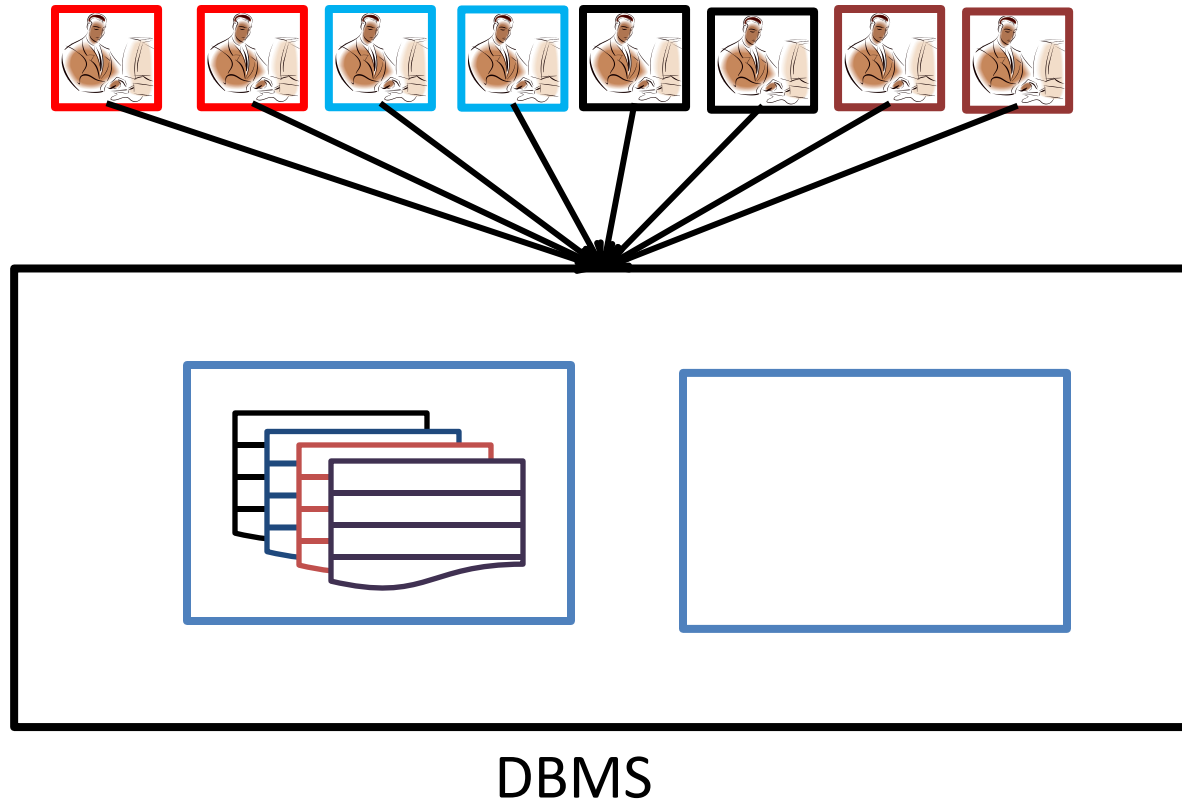
- Scale up and down system size on demand
 - Utilize peaks and troughs in load
- Minimize operating cost while ensuring good performance
 - A database system built over a pay-per-use infrastructure

Elasticity in the Database Layer



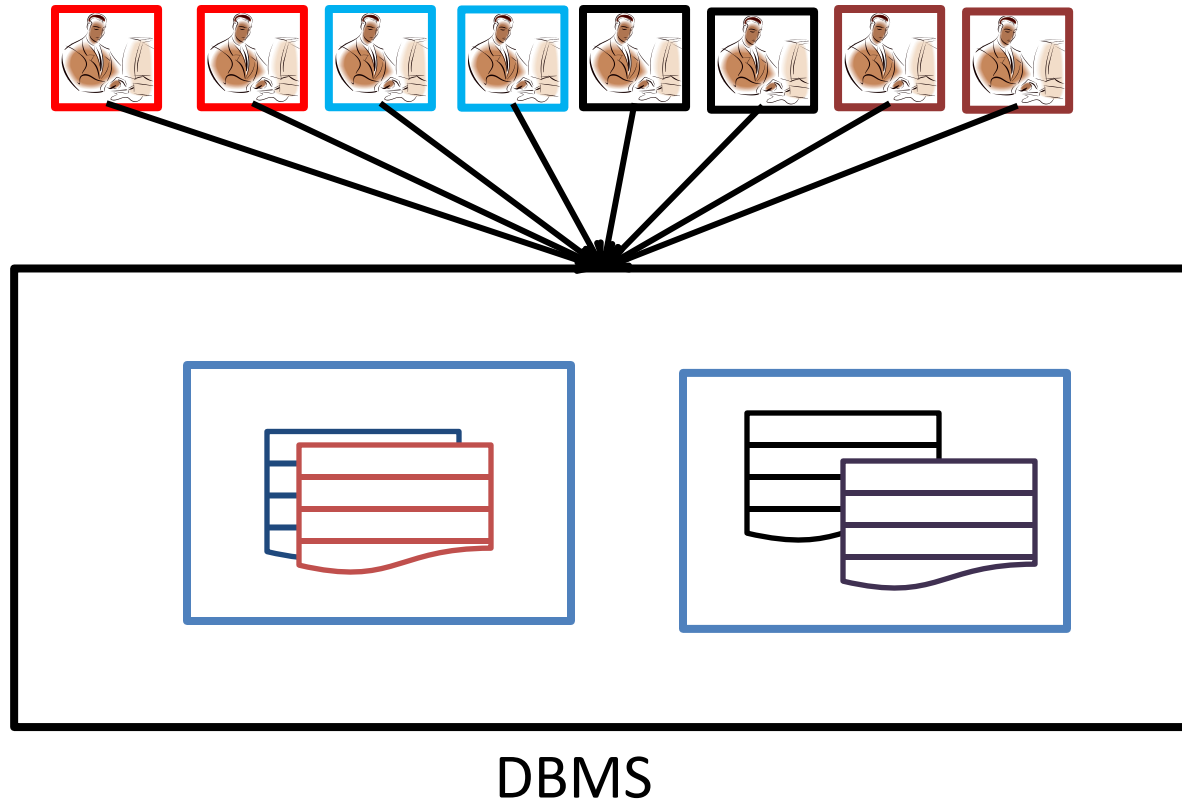
Elasticity in the Database Layer

Capacity expansion to deal with high load –
Guarantee good performance



Elasticity in the Database Layer

Consolidation during periods of low load –
Cost Minimization



Live Database Migration

A Critical operation for effective elasticity

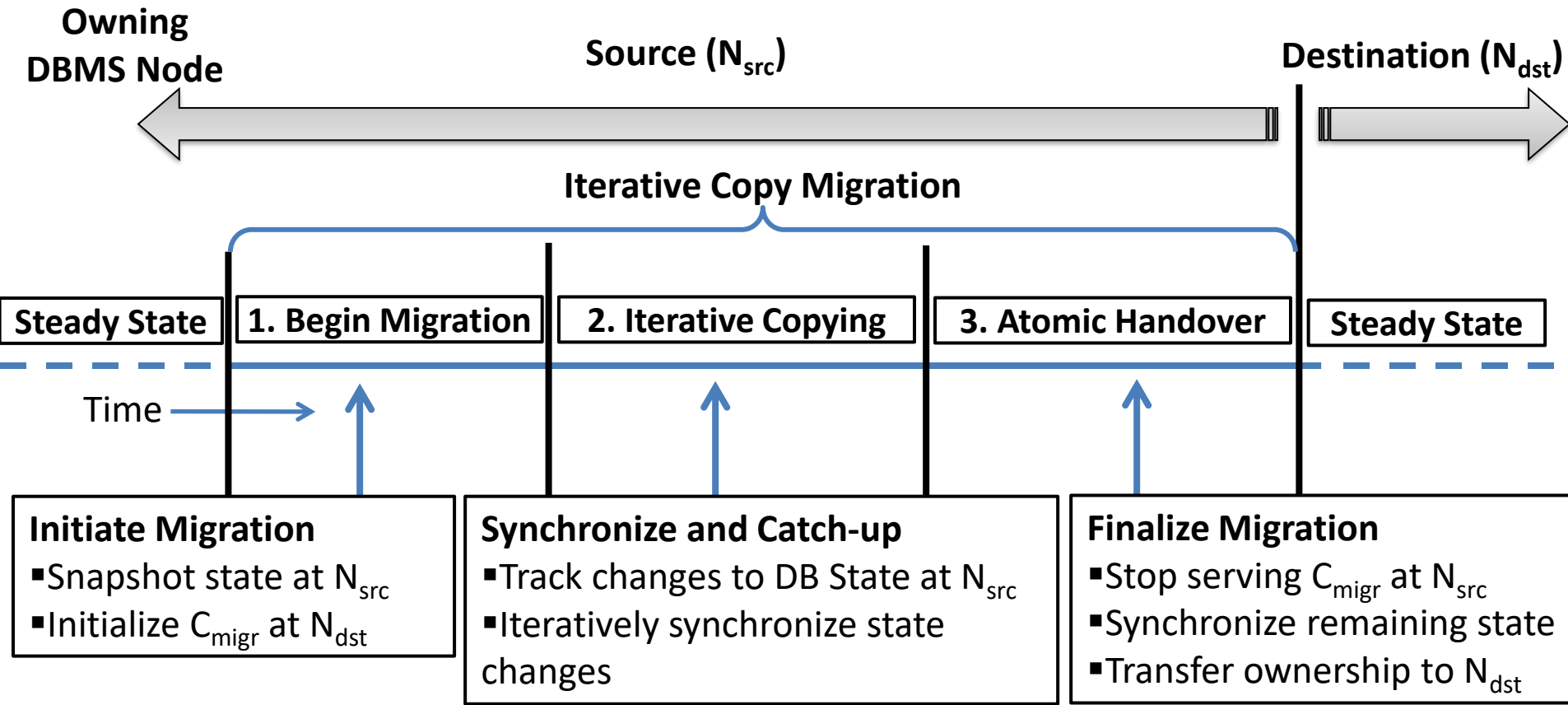
- Elasticity induced dynamics in a **Live** system
- **Minimal service interruption** for migrating data fragments
 - Minimize operations failing
 - Minimize unavailability window, if any
- Negligible **performance impact**
- **No overhead** during normal operation
- Guaranteed **safety** and **correctness**

Live Database Migration

Shared storage architecture [Das et al. Tech Report 2010]

- **Proactive** state migration
 - No need to migrate persistent data
 - Migrate database cache and transaction state proactively
 - Iteratively copy the state from source to destination
 - Ensure low impact on transaction latency and no aborted transactions

Migration in Shared Storage

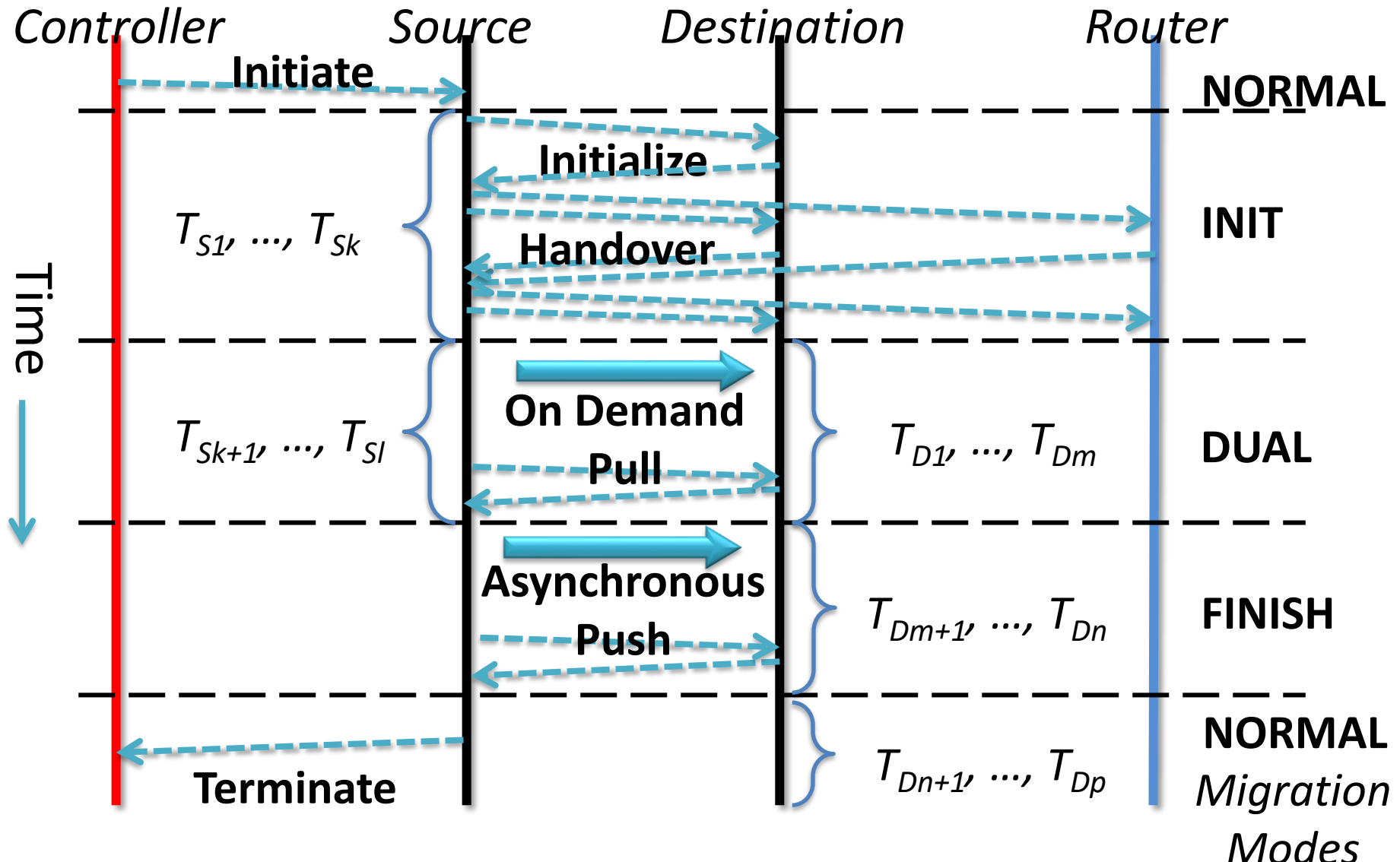


Live Database Migration

Shared nothing architecture [Elmore et al. SIGMOD 2011]

- **Reactive** state migration
 - Migrate minimal database state to the destination
 - Source and destination concurrently executing transactions
 - Synchronized DUAL mode
 - Source completes active transactions
 - Transfer ownership to the destination
 - Persistent image migrated asynchronously on demand

Migration in Shared Nothing



Research Challenges

Research Challenges (I)

- **Right sharing abstraction**
 - Shared table design popularly used for SaaS
 - Is this the right sharing model for PaaS?
 - Tenant isolation, both for security and performance
 - Supporting diverse schemas

Research Challenges (II)

- **High Availability, Failover and Load Balancing**
 - Large number of instances and databases
 - At the database level, or below the database
- **Distributed Fabric**
 - Manageability
 - Many different levels of failure detection
 - Scale out

Research Challenges (III)

- **Performance**

- Single tenant vs. multitenant
- Governance
- Benchmarks

- **Resource Models**

- Cost-efficiency
- Performance guarantees
- SLAs

Research Challenges (IV)

- **Balance functionality with scale**
 - Most tenants are small
 - The systems can potentially have hundreds of thousands of tenants
 - What are the right abstractions for this scale?
 - What functionality should be supported?

Research Challenges (V)

- **SLAs and Operating Cost as First-Class features**
 - Important to adhere to SLAs – tenants pays for these SLAs
 - Minimize the total operating cost – a new optimization goal in system design
 - Interplay between Cost minimization and SLA satisfaction

Concluding Remarks

- Data Management for Cloud Computing poses a **fundamental challenge** to database researchers:
 - Scalability
 - Reliability
 - Data Consistency
 - Elasticity
 - Differential Pricing
- **Radically different** approaches and solutions are warranted to overcome this challenge:
 - Need to understand the nature of **new applications**
- Novel Data Management Challenges coupled with Distributed and Parallel Computing issues

References

- **[Cooper et al., ACM SoCC 2010]** Benchmarking Cloud Serving Systems with YCSB, B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, R. Sears, In ACM SoCC 2010
- **[Brantner et al., SIGMOD 2008]** Building a Database on S3 by M. Brantner, D. Florescu, D. Graf, D. Kossman, T. Kraska, SIGMOD'08
- **[Kraska et al., VLDB 2009]** Consistency Rationing in the Cloud: Pay only when it matters, T. Kraska, M. Hentschel, G. Alonso, and D. Kossmann, VLDB 2009
- **[Lomet et al., CIDR 2009]** Unbundling Transaction Services in the Cloud, D. Lomet, A. Fekete, G. Weikum, M. Zwillig, CIDR'09
- **[Das et al., HotCloud 2009]** ElasTraS: An Elastic Transactional Data Store in the Cloud, S. Das, D. Agrawal, and A. El Abbadi, USENIX HotCloud, 2009
- **[Das et al., ACM SoCC 2010]** G-Store: A Scalable Data Store for Transactional Multi key Access in the Cloud, S. Das, D. Agrawal, and A. El Abbadi, ACM SOCC, 2010.
- **[Das et al., TR 2010]** ElasTraS: An Elastic, Scalable, and Self Managing Transactional Database for the Cloud, S. Das, S. Agarwal, D. Agrawal, and A. El Abbadi, UCSB Tech Report CS 2010-04

References/sources

- Divy Agrawal, Sudipto Das, and Amr El Abbadi. Big Data and Cloud Computing: Current State and Future Opportunities. EDBT 2011 Tutorial. Department of Computer Science. University of California at Santa Barbara.
- **[Yang et al., CIDR 2009]** A scalable data platform for a large number of small applications, F. Yang, J. Shanmugasundaram, and R. Yerneni, CIDR, 2009
- **[Kossmann et al., SIGMOD 2010]** An Evaluation of Alternative Architectures for Transaction Processing in the Cloud, D Kossmann, T. Kraska, Simon Loesing, In SIGMOD 2010
- **[Aulbach et al., SIGMOD 2009]** A Comparison of Flexible Schemas for Software as a Service, S. Aulbach, D. Jacobs, A. Kemper, M. Seibold, In SIGMOD 2009
- **[Aulbach et al., SIGMOD 2008]** Multi-Tenant Databases for Software as a Service: Schema and Mapping Techniques, In SIGMOD 2008
- **[Weissman et al., SIGMOD 2009]** The Design of the Force.com Multitenant Internet Application Development Platform, C.D. Weissman, S. Bobrowski, In SIGMOD 2009
- **[Jacobs et al., DTW 2007]** Ruminations of Multi-Tenant Databases, D. Jacobs, S. Aulbach, In DTW 2007
- **[Chang et al., OSDI 2006]** Bigtable: A Distributed Storage System for Structured Data, F. Chang et al., In OSDI 2006
- **[Cooper et al., VLDB 2008]** PNUTS: Yahoo!'s hosted data serving platform, B. F. Cooper et al., In VLDB 2008
- **[DeCandia et al., SOSP 2007]** Dynamo: amazon's highly available key-value store, G. DeCandia et al., In SOSP 2007